

Lab 3:

Title: Design and Implementation of Nondeterministic Finite Automata (NFA) for Various Languages.

Theory:

A Nondeterministic Finite Automaton (NFA) is a model of computation that, unlike a DFA, allows for ambiguity or "nondeterminism" in its transitions. From a single state and a single input symbol, an NFA can transition to zero, one, or multiple next states. This model is conceptually more flexible than a DFA, yet both have the same computational power—they both recognize the class of regular languages.

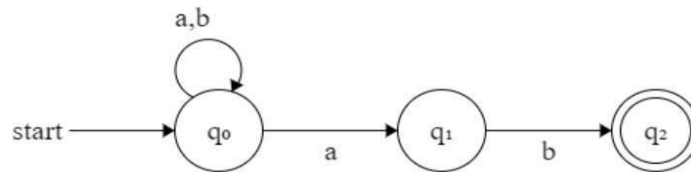
Components of an NFA

An NFA is formally defined as a 5-tuple: $(Q, \Sigma, \delta, q_0, F)$, which is very similar to a DFA, but with one crucial difference:

- Q is a finite set of states.
- Σ (Sigma) is a finite set of input symbols (the alphabet).
- δ (Delta) is the transition function. This is where the nondeterminism comes in. It maps a state and an input symbol to a set of next states, rather than a single state. Formally, $\delta: Q \times \Sigma \rightarrow 2Q$, where $2Q$ is the power set of Q (the set of all subsets of Q).
- q_0 is the start state, an element of Q .
- F is a set of final (or accepting) states, a subset of Q .

An additional feature of some NFAs is the epsilon (ϵ) transition, which allows the machine to move from one state to another without consuming an input symbol. This further simplifies the design of certain automata.

Program 1: Design and implementation of NFA to accept strings over $\{a, b\}$ that ends with ab .



Source code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>
bool isAccepted(char *str)
{
    int n = strlen(str);
    if (n < 2)
        return false;

    return (str[n - 2] == 'a' && str[n - 1] == 'b');
}
int main()
{
    char *input = NULL;
    char buffer[1000];
    printf("Program by: Mahesh Udas\n");

    printf("Enter a string over {a,b}: ");
    if (fgets(buffer, sizeof(buffer), stdin) == NULL)
    {
        printf("Error reading input.\n");
        return 1;
    }
    buffer[strcspn(buffer, "\n")] = '\0';
    input = strdup(buffer);
    if (isAccepted(input))
        printf("Accepted: String ends with 'ab'.\n");
    else
        printf("Rejected: String does not end with 'ab'.\n");
    free(input);
    return 0;
}
```

Output and discussion

Program by: Mahesh Udas

Enter a string over {a,b}: babbabab

Accepted: String ends with "ab".

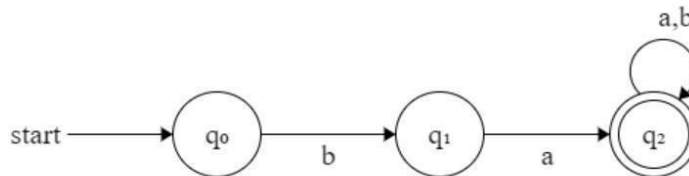
Program by: Mahesh Udas

Enter a string over {a,b}: bbabbabb

Rejected: String does not end with "ab".

This C program determines if a string ends with "ab". It checks the last two characters of the input string. The isAccepted function returns true if the string has at least two characters and they are 'a' and 'b' respectively, otherwise it returns false.

Program 2: Design and implementation of NFA to accept strings over {a, b} that starts with ba



Source code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>
```

```
bool isAccepted(char *str)
{
```

```
    int n = strlen(str);
    if (n < 2)
```

```
        return false;
    return (str[0] == 'b' && str[1] == 'a');
```

```
}
int main()
{
```

```
    char *input = NULL;
```

```
    char buffer[1000];
```

```
    printf("Program by: Mahesh Udas\n");
```

```
    printf("Enter a string over {a,b}: ");
```

```
    if (fgets(buffer, sizeof(buffer), stdin) == NULL)
    {
```

```
        printf("Error reading input.\n");
        return 1;
```

```
    }
    buffer[strcspn(buffer, "\n")] = '\0';
```

```
    input = strdup(buffer);
```

```
    if (isAccepted(input))
```

```
        printf("Accepted: String starts with \"ba\".\n");
```

```
    else
```

```
        printf("Rejected: String does not start with \"ba\".\n");
    free(input);
```

```
    return 0;
```

```
}
```

Output and discussion

Program by: Mahesh Udas

Enter a string over {a,b}: aaaba

Rejected: String does not start with "ba".

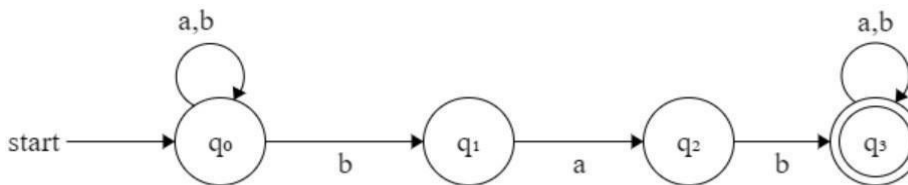
Program by: Mahesh Udas

Enter a string over {a,b}: baba

Accepted: String starts with "ba".

This C program checks if a string over {a, b} starts with "ba". It reads input using fgets, removes the newline, and duplicates the string. The isAccepted function returns true only if the first two characters are 'b' and 'a'. Based on this, the program prints whether the string is accepted or rejected, demonstrating simple pattern recognition.

Program 3: Design and Implementation of NFA to accept string over {a, b} such that each string contain bab as substring.



Source code:

```
#include <stdio.h>
#include <string.h>
#include <stdbool.h>

bool isAccepted(char *str)
{
    return strstr(str, "bab") != NULL;
}

int main()
{
    char buffer[1000];
    printf("Program by: Mahesh Udas\n");
    printf("Enter a string over {a,b}: ");
    if (fgets(buffer, sizeof(buffer), stdin) == NULL)
    {
        printf("Error reading input.\n");
        return 1;
    }

    buffer[strcspn(buffer, "\n")] = '\0';
    for (int i = 0; buffer[i] != '\0'; i++)
    {
        if (buffer[i] != 'a' && buffer[i] != 'b')
        {
            printf("Invalid input: only 'a' and 'b' are\n");
            printf("allowed.\n");
            return 1;
        }
    }
    if (isAccepted(buffer))
        printf("Accepted: String contains \"bab\" as\n");
        printf("substring.\n");
    else
        printf("Rejected: String does not contain \"bab\".\n");
    return 0;
}
```

Output and discussion

```
Program by: Mahesh Udas
Enter a string over {a,b}: aaba
Rejected: String does not contain "bab".
```

```
Program by: Mahesh Udas
Enter a string over {a,b}: aabab
Accepted: String contains "bab" as substring.
```

This C program validates input to ensure it solely comprises 'a' and 'b'. It then checks if the input string contains "bab" as a substring. The isAccepted function leverages strstr to efficiently locate the substring. If "bab" is found, the string is accepted; otherwise, it is rejected. Error handling for invalid characters and input reading is included.

Conclusion:

This lab explored the design and implementation of Nondeterministic Finite Automata (NFA) for recognizing various regular languages over the alphabet {a, b}. Three distinct NFA programs were developed to accept strings with specific patterns: ending with "ab", starting with "ba", and containing "bab" as a substring. Unlike DFAs, NFAs allow multiple transition paths from a single state, providing greater flexibility in design though both models recognize the same class of regular languages.